第3章 操 作 符

在最底层，Java中的数据是通过使用操作符来操做的。

Java是建立在C++基础之上的，所以C和C++程序员应该非常熟悉Java的大多数操作符。当然，Java也做了一些改进与简化。

如果读者熟悉C或C++的语法，那么只需快速浏览本章和下一章，看看Java与这些语言之间的差异。但是，如果读者觉得很难理解这两章的内容，那就请您先阅读可以从www.MindView.net上免费下载的多媒体课程《Thinking in C》，其中包括精心设计的有声讲解、幻灯片、练习以及解答，这些能带领读者快速掌握学习Java所必需的基础知识。

3.1 更简单的打印语句

在前一章中，我们介绍了Java的打印语句：

```
System.out.println("Rather a lot to type");
```

你可以看到，这条语句不仅涉及许多类型（因此有许多多余的连接），而且它读起来也颇为费劲。在Java之前和之后出现的大多数语言都已经采取了一种简单得多的方式来提供这种常用语句。

在第6章中将介绍静态导入（static import）这个在Java SE5中新增加的概念，并将创建一个小类库来简化打印语句的编写。但是，在开始使用这个类库之前，不必先去了解其中的细节。通过使用这个新类库，可以把上一章中的程序改写如下：

```
//: operators/HelloDate.java
import java.util.*;
import static net.mindview.util.Print.*;

public class HelloDate {
  public static void main(String[] args) {
    print("Hello, it's: ");
    print(new Date());
  }
} /* Output: (55% match)
Hello, it's:
Wed Oct 05 14:39:05 MDT 2005
*///:~
```


改写后的程序清爽了许多。请注意，我们在第二个**import**语句中插入了**static**关键字。

要想使用这个类库，必须从www.MindView.net或其镜像之一下载本书的代码包，然后解压代码目录树，并在你的计算机的CLASSPATH环境变量中添加该代码目录树的根目录。（你最终会获得有关类路径的完整介绍，但是你也应该尽早习惯它带来的麻烦。唉，它是你在使用Java时最常见的问题之一。）

尽管使用**net.mindview.util.Print**可以很好地简化大多数的代码，但是它并非在任何场合都显得很恰当。如果在代码中只有少量的打印语句，我还是先用**import**然后编写完整的**System.out.println()**。

练习1：(1) 使用“简短的”和正常的打印语句来编写一个程序。

3.2 使用Java操作符

操作符接受一个或多个参数，并生成一个新值。参数的形式与普通的方法调用不同，但效果是相同的。加号和一元的正号（+）、减号和一元的负号（-）、乘号（*）、除号（/）以及赋值号（=）的用法与其他编程语言类似。

操作符作用于操作数，生成一个新值。另外，有些操作符可能会改变操作数自身的值，这被称为“副作用”。那些能改变其操作数的操作符，最普遍的用途就是用来产生副作用；但要记住，使用此类操作符生成的值，与使用没有副作用的操作符生成的值，没有什么区别。 94

几乎所有的操作符都只能操作“基本类型”。例外的操作符是“=”、“==”和“!=”，这些操作符能操作所有的对象（这也是对象易令人糊涂的地方）。除此以外，**String**类支持“+”和“+=”。

3.3 优先级

当一个表达式中存在多个操作符时，操作符的优先级就决定了各部分的计算顺序。Java对计算顺序做了特别的规定。其中，最简单的规则就是先乘除后加减。程序员经常会忘记其他优先级规则，所以应该用括号明确规定计算顺序。例如，以下语句中的(1)和(2)：

```
//: operators/Precedence.java

public class Precedence {
  public static void main(String[] args) {
    int x = 1, y = 2, z = 3;
    int a = x + y - 2/2 + z;          // (1)
    int b = x + (y - 2)/(2 + z);      // (2)
    System.out.println("a = " + a + " b = " + b);
  }
} /* Output:
a = 5 b = 1
*///:~
```

这两个语句看起来大体相同，但是从输出就可以看出它们具有迥然不同的含义，而这正是使用括号的结果。

请注意，**System.out.println()**语句中包含“+”操作符。在这种上下文环境中，“+”意味着“字符串连接”，并且如果必要，它还要执行“字符串转换”。当编译器观察到一个**String**后面紧跟一个“+”，而这个“+”的后面又紧跟一个非**String**类型的元素时，就会尝试着将这个非String类型的元素转换为**String**。正如在输出中所看到的，它成功地将**a**和**b**从**int**转换为**String**了。

3.4 赋值

赋值使用操作符“=”。它的意思是“取右边的值（即右值），把它复制给左边（即左值）”。右值可以是任何常数、变量或者表达式（只要它能生成一个值就行）。但左值必须是一个明确的、已命名的变量。也就是说，必须有一个物理空间可以存储等号右边的值。举例来说，可将一个常数赋给一个变量： 95

```
a = 4;
```

但是不能把任何东西赋给一个常数，常数不能作为左值（比如不能说**4=a**；）。

对基本数据类型的赋值是很简单的。基本类型存储了实际的数值，而并非指向一个对象的引用，所以在为其赋值的时候，是直接将一个地方的内容复制到了另一个地方。例如，对基本数据类型使用**a=b**，那么**b**的内容就复制给**a**。若接着又修改了**a**，而**b**根本不会受这种修改的影响。

作为程序员，这正是大多数情况下我们所期望的。

但是在为对象“赋值”的时候，情况却发生了变化。对一个对象进行操作时，我们真正操作的是对对象的引用。所以倘若“将一个对象赋值给另一个对象”，实际是将“引用”从一个地方复制到另一个地方。这意味着假若对对象使用**c=d**，那么**c**和**d**都指向原本只有d指向的那个对象。下面这个例子将向大家阐示这一点。

```
//: operators/Assignment.java
// Assignment with objects is a bit tricky.
import static net.mindview.util.Print.*;

class Tank {
  int level;
}

public class Assignment {
  public static void main(String[] args) {
    Tank t1 = new Tank();
    Tank t2 = new Tank();
    t1.level = 9;
    t2.level = 47;
    print("1: t1.level: " + t1.level +
          ", t2.level: " + t2.level);
    t1 = t2;
    print("2: t1.level: " + t1.level +
          ", t2.level: " + t2.level);
    t1.level = 27;
    print("3: t1.level: " + t1.level +
          ", t2.level: " + t2.level);
  }
} /* Output:
1: t1.level: 9, t2.level: 47
2: t1.level: 47, t2.level: 47
3: t1.level: 27, t2.level: 27
*///:~
```

96

Tank类非常简单，它的两个实例（**t1**和**t2**）是在**main()**里创建的。对每个**Tank**类对象的**level**域都赋予了一个不同的值，然后，将**t2**赋给**t1**，接着又修改了**t1**。在许多编程语言中，我们可能会期望**t1**和**t2**总是相互独立的。但由于赋值操作的是一个对象的引用，所以修改**t1**的同时也改变了**t2**！这是由于**t1**和**t2**包含的是相同的引用，它们指向相同的对象。（原本**t1**包含的对对象的引用，是指向一个值为9的对象。在对**t1**赋值的时候，这个引用被覆盖，也就是丢失了；而那个不再被引用的对象会由“垃圾回收器”自动清理。）

这种特殊的现象通常称作“别名现象”，是Java操作对象的一种基本方式。在这个例子中，如果想避免别名问题应该怎么办呢？可以这样写：

```
t1.level = t2.level;
```

这样便可以保持两个对象彼此独立，而不是将**t1**和**t2**绑定到相同的对象。但你很快就会意识到，直接操作对象内的域容易导致混乱，并且，违背了良好的面向对象程序设计的原则。这可不是一个小问题，所以从现在开始大家就应该留意，为对象赋值可能会产生意想不到的结果。

练习2：(1) 创建一个包含一个**float**域的类，并用这个类来展示别名机制。

3.4.1 方法调用中的别名问题

将一个对象传递给方法时，也会产生别名问题：

```
//: operators/PassObject.java
// Passing objects to methods may not be
// what you're used to.
import static net.mindview.util.Print.*;
```

97

```
class Letter {
  char c;
}

public class PassObject {
  static void f(Letter y) {
    y.c = 'z';
  }
  public static void main(String[] args) {
    Letter x = new Letter();
    x.c = 'a';
    print("1: x.c: " + x.c);
    f(x); 
    print("2: x.c: " + x.c);
  }
} /* Output:
1: x.c: a
2: x.c: z
*///:~
```

在许多编程语言中，方法f()似乎要在它的作用域内复制其参数**Letter y**的一个副本；但实际上只是传递了一个引用。所以代码行

```
y.c = 'z';
```

实际改变的是f()之外的对象。

别名引起的问题及其解决办法是很复杂的话题，本书的在线补充材料涵盖了此话题。但是你现在就应该知道它的存在，并在使用中注意这个陷阱。

练习3：(1) 创建一个包含一个**float**域的类，并用这个类来展示方法调用时的别名机制。

3.5 算术操作符

Java的基本算术操作符与其他大多数程序设计语言是相同的。其中包括加号（+）、减号（−）、除号（/）、乘号（*）以及取模操作符（%，它从整数除法中产生余数）。整数除法会直接去掉结果的小数位，而不是四舍五入地圆整结果。

Java也使用一种来自C和C++的简化符号同时进行运算与赋值操作。这用操作符后紧跟一个等号来表示，它对于Java中的所有操作符都适用，只要其有实际意义就行。例如，要将**x**加4，并将结果赋回给**x**，可以这么写：**x+=4**。 98

下面这个例子展示了各种算术操作符的用法：

```
//: operators/MathOps.java
// Demonstrates the mathematical operators.
import java.util.*;
import static net.mindview.util.Print.*;

public class MathOps {
  public static void main(String[] args) {
    // Create a seeded random number generator:
    Random rand = new Random(47);
    int i, j, k;
    // Choose value from 1 to 100:
    j = rand.nextInt(100) + 1;
    print("j : " + j);
    k = rand.nextInt(100) + 1;
    print("k : " + k);
    i = j + k;
    print("j + k : " + i);
    i = j - k;
    print("j - k : " + i);
    i = k / j;
```

```
    print("k / j : " + i);
    i = k * j;
    print("k * j : " + i);
    i = k % j;
    print("k % j : " + i);
    j %= k;
    print("j %= k : " + j);
    // Floating-point number tests:
    float u, v, w; // Applies to doubles, too
    v = rand.nextFloat();
    print("v : " + v);
    w = rand.nextFloat();
    print("w : " + w);
    u = v + w;
    print("v + w : " + u);
    u = v - w;
    print("v - w : " + u);
    u = v * w;
    print("v * w : " + u);
    u = v / w;
    print("v / w : " + u);
    // The following also works for char,
    // byte, short, int, long, and double:
    u += v;
    print("u += v : " + u);
    u -= v;
    print("u -= v : " + u);
    u *= v;
    print("u *= v : " + u);
    u /= v;
    print("u /= v : " + u);
  }
} /* Output:
j : 59
k : 56
j + k : 115
j - k : 3
k / j : 0
k * j : 3304
k % j : 56
j %= k : 3
v : 0.5309454
w : 0.0534122
v + w : 0.5843576
v - w : 0.47753322
v * w : 0.028358962
v / w : 9.940527
u += v : 10.471473
u -= v : 9.940527
u *= v : 5.2778773
u /= v : 9.940527
*///:~
```

99

要生成数字，程序首先会创建一个**Random**类的对象。如果在创建过程中没有传递任何参数，那么Java就会将当前时间作为随机数生成器的种子，并由此在程序每一次执行时都产生不同的输出。但是，在本书的示例中，示例末尾所展示的输出都尽可能一致，这一点很重要，因为这样就使得这些输出可以用外部工具来验证。通过在创建**Random**对象时提供种子（用于随机数生成器的初始化值，随机数生成器对于特定的种子值总是产生相同的随机数序列），就可以在每一次执行程序时都生成相同的随机数，因此其输出是可验证的[⊖]。要想生成更多的各种不同的输出，可以随意移除本书示例中的种子。

100

⊖ 数字47在我加盟的一家学院里被认为是“魔幻数字”，至今仍是这样。

通过**Random**类的对象，程序可生成许多不同类型的随机数字。做法很简单，只需调用方法**nextInt()**和**nextFloat()**即可（也可以调用**nextLong()**或者**nextDouble()**）。传递给**nextInt()**的参数设置了所产生的随机数的上限，而其下限为0，但是这个下限并不是我们想要的，因为它会产生除0的可能性，因此我们对结果做了加1操作。

练习4：(2) 编写一个计算速度的程序，它所使用的距离和时间都是常量。

3.5.1 一元加、减操作符

一元减号（-）和一元加号（+）与二元减号和加号都使用相同的符号。根据表达式的书写形式，编译器会自动判断出使用的是哪一种。例如语句

```
x = -a;
```

的含义是显然的。编译器能正确识别下述语句：

```
x = a * -b;
```

但读者会被搞糊涂，所以有时更明确地写成：

```
x = a * (-b);
```

一元减号用于转变数据的符号，而一元加号只是为了与一元减号相对应，但是它唯一的作用仅仅是将较小类型的操作数提升为**int**。

3.6 自动递增和递减

和C类似，Java提供了大量的快捷运算。这些快捷运算使编码更方便，同时也使得代码更容易阅读，但是有时可能使代码阅读起来更困难。

递增和递减运算是两种相当不错的快捷运算（常称为“自动递增”和“自动递减”运算）。其中，递减操作符是“--”，意为“减少一个单位”；递增操作符是“++”，意为“增加一个单位”。举个例子来说，假设**a**是一个**int**（整数）值，则表达式**++a**就等价于（**a = a + 1**）。递增和递减操作符不仅改变了变量，并且以变量的值作为生成的结果。 101

这两个操作符各有两种使用方式，通常称为“前缀式”和“后缀式”。“前缀递增”表示“++”操作符位于变量或表达式的前面；而“后缀递增”表示“++”操作符位于变量或表达式的后面。类似地，“前缀递减”意味着“--”操作符位于变量或表达式的前面；而“后缀递减”意味着“--”操作符位于变量或表达式的后面。对于前缀递增和前缀递减（如 **++a** 或 **--a**），会先执行运算，再生成值。而对于后缀递增和后缀递减（如**a++**或**a--**），会先生成值，再执行运算。下面是一个例子：

```
//: operators/AutoInc.java
// Demonstrates the ++ and -- operators.
import static net.mindview.util.Print.*;

public class AutoInc {
  public static void main(String[] args) {
    int i = 1;
    print("i : " + i);
    print("++i : " + ++i); // Pre-increment
    print("i++ : " + i++); // Post-increment
    print("i : " + i);
    print("--i : " + --i); // Pre-decrement
    print("i-- : " + i--); // Post-decrement
    print("i : " + i);
  }
} /* Output:
i : 1
++i : 2
i++ : 2
```

```
i : 3
--i : 2
i-- : 2
i : 1
*///:~
```

从中可以看到，对于前缀形式，我们在执行完运算后才得到值。但对于后缀形式，则是在运算执行之前就得到值。它们是除那些涉及赋值的操作符以外，唯一具有“副作用”的操作符。也就是说，它们会改变操作数，而不仅仅是使用自己的值。

102

递增操作符正是对C++这个名字的一种解释，暗示着“超越C一步”。在早期的一次有关Java的演讲中，Bill Joy（Java创始人之一）声称“Java=C++--”（C加加减减）意味着Java已去除了C++中一些很困难而又没必要的东西，成为了一种更精简的语言。正如大家会在这本书中学到的，Java的许多地方更精简了，然而并不是说Java在其他方面也比C++ 容易很多。

3.7 关系操作符

关系操作符生成的是一个**boolean**（布尔）结果，它们计算的是操作数的值之间的关系。如果关系是真实的，关系表达式会生成**true**（真）；如果关系不真实，则生成**false**（假）。关系操作符包括小于（<）、大于（>）、小于或等于（<=）、大于或等于（>=）、等于（==）以及不等于（!=）。等于和不等于适用于所有的基本数据类型，而其他比较符不适用于**boolean**类型。因为**boolean**值只能为**true**或**false**，“大于”和“小于”没有实际意义。

3.7.1 测试对象的等价性

关系操作符==和!=也适用于所有对象，但这两个操作符通常会使第一次接触Java的程序员感到迷惑。下面是一个例子：

```
//: operators/Equivalence.java

public class Equivalence {
  public static void main(String[] args) {
    Integer n1 = new Integer(47);
    Integer n2 = new Integer(47);
    System.out.println(n1 == n2);
    System.out.println(n1 != n2);
  }
} /* Output:
false
true
*///:~
```

语句**System.out.println(n1 == n2)**将打印出括号内的比较式的布尔值结果。读者可能认为输出结果肯定先是true，再是false，因为两个**Integer**对象都是相同的。但是尽管对象的内容相同，然而对象的引用却是不同的，而==和!=比较的就是对象的引用。所以输出结果实际上先是false，再是true。这自然会使第一次接触关系操作符的人感到惊奇。

103

如果想比较两个对象的实际内容是否相同，又该如何操作呢？此时，必须使用所有对象都适用的特殊方法**equals()**。但这个方法不适用于“基本类型”，基本类型直接使用==和!=即可。下面举例说明如何使用：

104

```
//: operators/EqualsMethod.java

public class EqualsMethod {
  public static void main(String[] args) {
    Integer n1 = new Integer(47);
    Integer n2 = new Integer(47);
    System.out.println(n1.equals(n2));
```

```
  }
} /* Output:
true
*///:~
```

结果正如我们所预料的那样。但事情并不总是这么简单！假设你创建了自己的类，就像下面这样：

```
//: operators/EqualsMethod2.java
// Default equals() does not compare contents.

class Value {
  int i;
}

public class EqualsMethod2 {
  public static void main(String[] args) {
    Value v1 = new Value();
    Value v2 = new Value();
    v1.i = v2.i = 100;
    System.out.println(v1.equals(v2));
  }
} /* Output:
false
*///:~
```

事情再次变得令人费解了：结果又是**false**！这是由于**equals()**的默认行为是比较引用。所以除非在自己的新类中覆盖**equals()**方法，否则不可能表现出我们希望的行为。

遗憾的是，我们要到第7章才学习覆盖，到第17章才学习如何恰当地定义**equals()**。但在这之前，请留意**equals()**的这种行为表现方式，这样或许能够避免一些“灾难”。

大多数Java类库都实现了**equals()**方法，以便用来比较对象的内容，而非比较对象的引用。

练习5：(2) 创建一个名为**Dog**的类，它包含两个**String**域：**name**和**says**。在**main()**方法中，创建两个Dog对象，一个名为spot（它的叫声为“Ruff！”），另一个名为scruffy（它的叫声为“Wurf！”）。然后显示它们的名字和叫声。

练习6：(3) 在练习5的基础上，创建一个新的**Dog**索引，并对其赋值为spot对象。测试用**==**和**equals()**方法来比较所有引用的结果。

3.8 逻辑操作符

逻辑操作符“与”（&&）、“或”（||）、“非”（!）能根据参数的逻辑关系，生成一个布尔值（**true**或**false**）。下面这个例子就使用了关系操作符和逻辑操作符。

```
//: operators/Bool.java
// Relational and logical operators.
import java.util.*;
import static net.mindview.util.Print.*;

public class Bool {
  public static void main(String[] args) {
    Random rand = new Random(47);
    int i = rand.nextInt(100);
    int j = rand.nextInt(100);
    print("i = " + i);
    print("j = " + j);
    print("i > j is " + (i > j));
    print("i < j is " + (i < j));
    print("i >= j is " + (i >= j));
    print("i <= j is " + (i <= j));
    print("i == j is " + (i == j));
    print("i != j is " + (i != j));
```

105

```
    // Treating an int as a boolean is not legal Java:
//! print("i && j is " + (i && j));
//! print("i || j is " + (i || j));
//! print("!i is " + !i);
    print("(i < 10) && (j < 10) is "
       + ((i < 10) && (j < 10)) );
    print("(i < 10) || (j < 10) is "
       + ((i < 10) || (j < 10)) );
  }
} /* Output:
i = 58
j = 55
i > j is true
i < j is false
i >= j is true
i <= j is false
i == j is false
i != j is true
(i < 10) && (j < 10) is false
(i < 10) || (j < 10) is false
*///:~
```

“与”、“或”、“非”操作只可应用于布尔值。与在C及C++中不同的是：不可将一个非布尔值当作布尔值在逻辑表达式中使用。在前面的代码中用“//!”注释掉的语句，就是错误的用法（这种注释语法使得注释能够被自动移除以方便测试）。后面的两个表达式先使用关系比较运算，生成布尔值，然后再对产生的布尔值进行逻辑运算。

注意，如果在应该使用**String**值的地方使用了布尔值，布尔值会自动转换成适当的文本形式。

在上述程序中，可将整数类型替换成除布尔型以外的其他任何基本数据类型。但要注意，对浮点数的比较是非常严格的。即使一个数仅在小数部分与另一个数存在极微小的差异，仍然认为它们是“不相等”的。即使一个数只比零大一点点，它仍然是“非零”值。

练习7：(3)编写一个程序，模拟扔硬币的结果。

3.8.1 短路

当使用逻辑操作符时，我们会遇到一种“短路”现象。即一旦能够明确无误地确定整个表达式的值，就不再计算表达式余下部分了。因此，整个逻辑表达式靠后的部分有可能不会被运算。下面是演示短路现象的例子：

106

```
//: operators/ShortCircuit.java
// Demonstrates short-circuiting behavior
// with logical operators.
import static net.mindview.util.Print.*;

public class ShortCircuit {
  static boolean test1(int val) {
    print("test1(" + val + ")");
    print("result: " + (val < 1));
    return val < 1;
  }
  static boolean test2(int val) {
    print("test2(" + val + ")");
    print("result: " + (val < 2));
    return val < 2;
  }
  static boolean test3(int val) {
    print("test3(" + val + ")");
    print("result: " + (val < 3));
    return val < 3;
  }
```

```
  public static void main(String[] args) {
    boolean b = test1(0) && test2(2) && test3(2);
    print("expression is " + b);
  }
} /* Output:
test1(0)
result: true
test2(2)
result: false
expression is false
*///:~
```

每个测试都会比较参数，并返回**true**或**false**。它也会打印信息告诉你正在调用测试。这些测试都作用于下面这个表达式：

```
test1(0) && test2(2) && test3(2)
```

你会很自然地认为所有这三个测试都会得以执行。但输出显示却并非这样。第一个测试生成结果**true**，所以表达式计算会继续下去。然而，第二个测试产生了一个**false**结果。由于这意味着整个表达式肯定为**false**，所以没必要继续计算剩余的表达式，那样只是浪费。“短路”一词的由来正源于此。事实上，如果所有的逻辑表达式都有一部分不必计算，那将获得潜在的性能提升。 107

3.9 直接常量

一般说来，如果在程序里使用了“直接常量”，编译器可以准确地知道要生成什么样的类型，但有时候却是模棱两可的。如果发生这种情况，必须对编译器加以适当的“指导”，用与直接量相关的某些字符来额外增加一些信息。下面这段代码向大家展示了这些字符。

```
//: operators/Literals.java
import static net.mindview.util.Print.*;

public class Literals {
  public static void main(String[] args) {
    int i1 = 0x2f; // Hexadecimal (lowercase)
    print("i1: " + Integer.toBinaryString(i1));
    int i2 = 0X2F; // Hexadecimal (uppercase)
    print("i2: " + Integer.toBinaryString(i2));
    int i3 = 0177; // Octal (leading zero)
    print("i3: " + Integer.toBinaryString(i3));
    char c = 0xffff; // max char hex value
    print("c: " + Integer.toBinaryString(c));
    byte b = 0x7f; // max byte hex value
    print("b: " + Integer.toBinaryString(b));
    short s = 0x7fff; // max short hex value
    print("s: " + Integer.toBinaryString(s));
    long n1 = 200L; // long suffix
    long n2 = 200l; // long suffix (but can be confusing)
    long n3 = 200;
    float f1 = 1;
    float f2 = 1F; // float suffix
    float f3 = 1f; // float suffix
    double d1 = 1d; // double suffix
    double d2 = 1D; // double suffix
    // (Hex and Octal also work with long)
  }
} /* Output:
i1: 101111
i2: 101111
i3: 1111111
c: 1111111111111111
b: 1111111
s: 111111111111111
*///:~
```

108

直接常量后面的后缀字符标志了它的类型。若为大写（或小写）的**L**，代表**long**（但是，使用小写字母**l**容易造成混淆，因为它看起来很像数字**1**）。大写（或小写）字母**F**，代表**float**；大写（或小写）字母**D**，则代表**double**。

十六进制数适用于所有整数数据类型，以前缀**0x**（或**0X**），后面跟随**0-9**或小写（或大写）的**a-f**来表示。如果试图将一个变量初始化成超出自身表示范围的值（无论这个值的数值形式如何），编译器都会向我们报告一条错误信息。注意在前面的代码中，已经给出了**char**、**byte**以及**short**所能表示的最大的十六进制值。如果超出范围，编译器会将值自动转换成**int**型，并告诉我们需要对这次赋值进行“窄化转型”（转型将在本章稍后部分定义）。这样我们就可清楚地知道自己的操作是否越界了。

八进制数由前缀0以及后续的0～7的数字来表示。

在C、C++或者Java中，二进制数没有直接常量表示方法。但是，在使用十六进制和八进制记数法时，以二进制形式显示结果将非常有用。通过使用**Integer**和**Long**类的静态方法**toBinaryString()**可以很容易地实现这一点。请注意，如果将比较小的类型传递给**Integer.toBinaryString()**方法，则该类型将自动被转换为int。

练习8：(2) 展示用十六进制和八进制记数法来操作long值，用**Long.toBinaryString()**来显示结果。

3.9.1 指数记数法

Java采用了一种很不直观的记数法来表示指数，例如：

```
//: operators/Exponents.java
// "e" means "10 to the power."

public class Exponents {
  public static void main(String[] args) {
    // Uppercase and lowercase 'e' are the same:
    float expFloat = 1.39e-43f;
    expFloat = 1.39E-43f;
    System.out.println(expFloat);
    double expDouble = 47e47d; // 'd' is optional
    double expDouble2 = 47e47; // Automatically double
    System.out.println(expDouble);
  }
} /* Output:
1.39E-43
4.7E48
*///:~
```

109

在科学与工程领域，“e”代表自然对数的基数，约等于2.718（Java中的**Math.E**给出了更精确的**double**型的值）。例如$1.39 \times e^{-43}$这样的指数表达式意味着1.39×2.718^{-43}。然而，设计FORTRAN语言的时候，设计师们很自然地决定e代表“10的幂次”。这种做法很奇怪，因为FORTRAN最初是面向科学与工程设计领域的，它的设计者们对引入这样容易令人混淆的概念应该很敏感才对[⊖]。但不管怎样，这种惯例在C、C++以及Java中被保留了下来。所以倘若习惯将e

⊖ John Kirkham写道：“我开始用计算机是在1962年，使用的是IBM 1620机器上的FORTRAN II。那时候，从60年代到70年代，FORTRAN一直都是使用大写字母。之所以会出现这一情况，可能是由于早期的输入设备大多是老式电传打字机，使用5位Baudot码，那是不包括小写字母的。乘幂表达式中的“E”也肯定是大写的，所以不会与自然对数的基数“e”发生冲突，后者必然是小写的。“E”这个字母的含义其实很简单，就是“Exponential”的意思，即“指数”或“幂数”，代表数字系统的基数——一般都是10。当时，八进制也在程序员中广泛使用。尽管我自己未看到它的使用，但假若我在乘幂表达式中看到一个八进制数字，我就会认为基数是8。我记得第一次看到用小写“e”表示指数是在70年代末期。我当时也觉得它极易产生混淆。产生这个问题是因为FORTRAN已经渐渐引入了小写字母，但并非一开始就有。如果你真的想使用自然对数的基数，有现成的函数可供利用，但它们都是大写的。”

作为自然对数的基数使用，那么在Java中看到像**1.39e**$^{-43}$**f**这样的表达式时，请转换思维，它真正的含义是1.39×10^{-43}。

注意如果编译器能够正确地识别类型，就不必在数值后附加字符。例如语句

```
long n3 = 200;
```

它不存在含混不清的地方，所以200后面的**L**是用不着的。然而，对于语句 110

```
float f4 = 1e-43f; // 10 to the power
```

编译器通常会将指数作为双精度数（double）处理，所以假如没有这个尾随的**f**，就会收到一条出错提示，告诉我们必须使用类型转换将**double**转换成**float**。

练习9：(1) 分别显示用**float**和**double**指数记数法所能表示的最大和最小的数字。

3.10 按位操作符

按位操作符用来操作整数基本数据类型中的单个“比特”（bit），即二进制位。按位操作符会对两个参数中对应的位执行布尔代数运算，并最终生成一个结果。

按位操作符来源于C语言面向底层的操作，在这种操作中经常需要直接操纵硬件，设置硬件寄存器内的二进制位。Java的设计初衷是嵌入电视机机顶盒内，所以这种面向底层的操作仍被保留了下来。但是，人们可能不会过多地用到位操作符。

如果两个输入位都是1，则按位“与”操作符（&）生成一个输出位1；否则生成一个输出位0。如果两个输入位里只要有一个是1，则按位“或”操作符（|）生成一个输出位1；只有在两个输入位都是0的情况下，它才会生成一个输出位0。如果输入位的某一个是1，但不全都是1，那么按位“异或”操作（^）生成一个输出位1。按位“非”（~），也称为取反操作符，它属于一元操作符，只对一个操作数进行操作（其他按位操作符是二元操作符）。按位“非”生成与输入位相反的值——若输入0，则输出1；若输入1，则输出0。

按位操作符和逻辑操作符都使用了同样的符号，因此我们能方便地记住它们的含义：由于位是非常“小”的，所以按位操作符仅使用了一个字符。

按位操作符可与等号（=）联合使用，以便合并运算和赋值：**&=**、**|=**和**^=**都是合法的（由于“~”是一元操作符，所以不可与“=”联合使用）。

我们将布尔类型作为一种单比特值对待，所以它多少有些独特。我们可对它执行按位“与”、按位“或”和按位“异或”运算，但不能执行按位“非”（大概是为了避免与逻辑NOT混淆）。对于布尔值，按位操作符具有与逻辑操作符相同的效果，只是它们不会中途“短路”。此外，针对布尔值进行的按位运算为我们新增了一个“异或”逻辑操作符，它并未包括在“逻辑”操作符的列表中。在移位表达式中，不能使用布尔运算，原因将在后面解释。 111

练习10：(3) 编写一个具有两个常量值的程序，一个具有交替的二进制位1和0，其中最低有效位为0，另一个也具有交替的二进制位1和0，但是其最低有效位为1（提示：使用十六进制常量来表示是最简单的方法）。取这两个值，用按位操作符以所有可能的方式结合运算它们，然后用**Integer.toBinaryString()**显示。

3.11 移位操作符

移位操作符操作的运算对象也是二进制的“位”。移位操作符只可用来处理整数类型（基本类型的一种）。左移位操作符（<<）能按照操作符右侧指定的位数将操作符左边的操作数向左移动（在低位补0）。“有符号”右移位操作符（>>）则按照操作符右侧指定的位数将操作符左边的

操作数向右移动。“有符号”右移位操作符使用“符号扩展”：若符号为正，则在高位插入0；若符号为负，则在高位插入1。Java中增加了一种“无符号”右移位操作符（>>>），它使用“零扩展”：无论正负，都在高位插入0。这一操作符是C或C++中所没有的。

如果对**char**、**byte**或者**short**类型的数值进行移位处理，那么在移位进行之前，它们会被转换为**int**类型，并且得到的结果也是一个**int**类型的值。只有数值右端的低5位才有用。这样可防止我们移位超过**int**型值所具有的位数。（译注：因为2的5次方为32，而int型值只有32位。）若对一个**long**类型的数值进行处理，最后得到的结果也是**long**。此时只会用到数值右端的低6位，以防止移位超过**long**型数值具有的位数。

“移位”可与“等号”（<<=或>>=或>>>=）组合使用。此时，操作符左边的值会移动由右边的值指定的位数，再将得到的结果赋给左边的变量。但在进行“无符号”右移位结合赋值操作时，可能会遇到一个问题：如果对**byte**或**short**值进行这样的移位运算，得到的可能不是正确的结果。它们会先被转换成**int**类型，再进行右移操作，然后被截断，赋值给原来的类型，在这种情况下可能得到−1的结果。下面这个例子演示了这种情况：

112

```
//: operators/URShift.java
// Test of unsigned right shift.
import static net.mindview.util.Print.*;

public class URShift {
  public static void main(String[] args) {
    int i = -1;
    print(Integer.toBinaryString(i));
    i >>>= 10;
    print(Integer.toBinaryString(i));
    long l = -1;
    print(Long.toBinaryString(l));
    l >>>= 10;
    print(Long.toBinaryString(l));
    short s = -1;
    print(Integer.toBinaryString(s));
    s >>>= 10;
    print(Integer.toBinaryString(s));
    byte b = -1;
    print(Integer.toBinaryString(b));
    b >>>= 10;
    print(Integer.toBinaryString(b));
    b = -1;
    print(Integer.toBinaryString(b));
    print(Integer.toBinaryString(b>>>10));
  }
} /* Output:
11111111111111111111111111111111
11111111111111111111
1111111111111111111111111111111111111111111111111111111111111111
11111111111111111111111111111111111111111111111111111111
11111111111111111111111111111111
11111111111111111111111111111111
11111111111111111111111111111111
11111111111111111111111111111111
11111111111111111111111111111111
11111111111111111111
*///:~
```

113

在最后一个移位运算中，结果没有赋给**b**，而是直接打印出来，所以其结果是正确的。下面这个例子向大家阐示了如何应用涉及“按位”操作的所有操作符：

```
//: operators/BitManipulation.java
// Using the bitwise operators.
import java.util.*;
import static net.mindview.util.Print.*;
```

```
public class BitManipulation {
  public static void main(String[] args) {
    Random rand = new Random(47);
    int i = rand.nextInt();
    int j = rand.nextInt();
    printBinaryInt("-1", -1);
    printBinaryInt("+1", +1);
    int maxpos = 2147483647;
    printBinaryInt("maxpos", maxpos);
    int maxneg = -2147483648;
    printBinaryInt("maxneg", maxneg);
    printBinaryInt("i", i);
    printBinaryInt("~i", ~i);
    printBinaryInt("-i", -i);
    printBinaryInt("j", j);
    printBinaryInt("i & j", i & j);
    printBinaryInt("i | j", i | j);
    printBinaryInt("i ^ j", i ^ j);
    printBinaryInt("i << 5", i << 5);
    printBinaryInt("i >> 5", i >> 5);
    printBinaryInt("(~i) >> 5", (~i) >> 5);
    printBinaryInt("i >>> 5", i >>> 5);
    printBinaryInt("(~i) >>> 5", (~i) >>> 5);

    long l = rand.nextLong();
    long m = rand.nextLong();
    printBinaryLong("-1L", -1L);
    printBinaryLong("+1L", +1L);
    long ll = 9223372036854775807L;
    printBinaryLong("maxpos", ll);
    long lln = -9223372036854775808L;
    printBinaryLong("maxneg", lln);
    printBinaryLong("l", l);
    printBinaryLong("~l", ~l);
    printBinaryLong("-l", -l);
    printBinaryLong("m", m);
    printBinaryLong("l & m", l & m);
    printBinaryLong("l | m", l | m);
    printBinaryLong("l ^ m", l ^ m);
    printBinaryLong("l << 5", l << 5);
    printBinaryLong("l >> 5", l >> 5);
    printBinaryLong("(~l) >> 5", (~l) >> 5);
    printBinaryLong("l >>> 5", l >>> 5);
    printBinaryLong("(~l) >>> 5", (~l) >>> 5);
  }
  static void printBinaryInt(String s, int i) {
    print(s + ", int: " + i + ", binary:\n   " +
      Integer.toBinaryString(i));
  }
  static void printBinaryLong(String s, long l) {
    print(s + ", long: " + l + ", binary:\n    " +
      Long.toBinaryString(l));
  }
} /* Output:
-1, int: -1, binary:
   11111111111111111111111111111111
+1, int: 1, binary:
   1
maxpos, int: 2147483647, binary:
   1111111111111111111111111111111
maxneg, int: -2147483648, binary:
   10000000000000000000000000000000
i, int: -1172028779, binary:
   10111010001001000100001010010101
~i, int: 1172028778, binary:
   1000101110110111011110101101010
-i, int: 1172028779, binary:
```

114

```
   10001011101101110111101011011011
j, int: 1717241110, binary:
   11001100101101100000101000101110
i & j, int: 570425364, binary:
   10001000000000000000000000010100
i | j, int: -25213033, binary:
   11111110011111110100011110010111
i ^ j, int: -595638397, binary:
   11011100011111110100011110000011
i << 5, int: 1149784736, binary:
   10001001000100001010010101000000
i >> 5, int: -36625900, binary:
   11111101110100010010001000010100
(~i) >> 5, int: 36625899, binary:
   10001011101101110111101011
i >>> 5, int: 97591828, binary:
   10111010001001000100001010100
(~i) >>> 5, int: 36625899, binary:
   10001011101101110111101011
...
*///:~
```

115

程序末尾调用了两个方法：**printBinaryInt()**和**printBinaryLong()**。它们分别接受**int**或**long**型的参数，并用二进制格式输出，同时附有简要的说明文字。上面的例子还展示了对**int**和**long**的所有按位操作符的作用，还展示了**int**和**long**的最小值、最大值、+1和−1值，以及它们的二进制形式，以使大家了解它们在机器中的具体形式。注意最高位表示符号：0为正，1为负。关于**int**部分的输出正如上面所示。

数字的二进制表示形式称为“有符号的二进制补码”。

练习11：(3) 以一个最高有效位为1的二进制数字开始（提示：使用十六进制常量），用有符号右移操作符对其进行右移，直至所有的二进制位都被移出为止，每移一位都要使用**Integer.toBinaryString()**显示结果。

练习12：(3) 以一个所有位都为1的二进制数字开始，先左移它，然后用无符号右移操作符对其进行右移，直至所有的二进制位都被移出为止，每移一位都要使用**Integer.toBinaryString()**显示结果。

练习13：(1) 编写一个方法，它以二进制形式显示**char**类型的值。使用多个不同的字符来展示它。

3.12 三元操作符 if-else

三元操作符也称为条件操作符，它显得比较特别，因为它有三个操作数；但它确实属于操作符的一种，因为它最终也会生成一个值，这与本章下一节中介绍的普通**if-else**语句是不同的。其表达式采取下述形式：

116

```
boolean-exp ? value0 : value1
```

如果boolean-exp（布尔表达式）的结果为**true**，就计算value0，而且这个计算结果也就是操作符最终产生的值。如果boolean-exp的结果为**false**，就计算value1，同样，它的结果也就成为了操作符最终产生的值。

当然，也可以换用普通的**if-else**语句（在后面介绍），但三元操作符更加简洁。尽管C（C中发明了该操作符）引以为傲的就是它是一种简练的语言，而且三元操作符的引入多半就是为了体现这种高效率的编程，但假如你打算频繁使用它，还是要多作思量，因为它很容易产生可读性极差的代码。

条件操作符与**if-else**完全不同，因为它会产生一个值。下面是这两者进行比较的示例：

```
//: operators/TernaryIfElse.java
import static net.mindview.util.Print.*;

public class TernaryIfElse {
  static int ternary(int i) {
    return i < 10 ? i * 100 : i * 10;
  }
  static int standardIfElse(int i) {
    if(i < 10)
      return i * 100;
    else
      return i * 10;
  }
  public static void main(String[] args) {
    print(ternary(9));
    print(ternary(10));
    print(standardIfElse(9));
    print(standardIfElse(10));
  }
} /* Output:
900
100
900
100
*///:~
```

可以看出，上面的**ternary()**中的代码与**standardIfElse()**中不用三元操作符的代码相比，显得更加紧凑；但**standardIfElse()**更易理解，而且不需要太多的录入。所以在选择使用三元操作符时，请务必仔细考虑。 117

3.13 字符串操作符 + 和 +=

这个操作符在Java中有一项特殊用途：连接不同的字符串。这一点已经在前面的例子中展示过了。尽管与 + 和 += 的传统使用方式不太一样，但我们还是很自然地使用这些操作符来做这件事情。

这项功能用在C++中似乎是个不错的主意，所以引入了*操作符重载*（operator overloading）机制，以便C++程序员可以为几乎所有操作符增加功能。但非常遗憾，与C++的另外一些限制结合在一起，使得操作符重载成为了一种非常复杂的特性，程序员在设计自己的类时必须对此有非常周全的考虑。与C++相比，尽管操作符重载在Java中更易实现（就像在C#语言中所展示的那样，它具有相当简单直接的操作符重载机制），但仍然过于复杂。所以Java程序员不能像C++和C# 程序员那样实现自己的重载操作符。

字符串操作符有一些很有趣的行为。如果表达式以一个字符串起头，那么后续所有操作数都必须是字符串型（请记住，编译器会把双引号内的字符序列自动转成字符串）：

```
//: operators/StringOperators.java
import static net.mindview.util.Print.*;

public class StringOperators {
  public static void main(String[] args) {
    int x = 0, y = 1, z = 2;
    String s = "x, y, z ";
    print(s + x + y + z);
    print(x + " " + s); // Converts x to a String
    s += "(summed) = "; // Concatenation operator
    print(s + (x + y + z));
    print("" + x); // Shorthand for Integer.toString()
  }
} /* Output:
x, y, z 012
```

118

```
0 x, y, z
x, y, z (summed) = 3
0
*///:~
```

请注意，第一个打印语句的输出是**012**而不是**3**，而**3**正是将这些整数求和之后应该得到的结果，之所以出现这种情况，是因为Java编译器会将**x**、**y**和**z**转换成它们的字符串形式，然后连接这些字符串，而不是先把它们加到一起。第二个打印语句把先导的变量转换为**String**，因此这个字符串转换将不依赖于第一个变量的类型。最后，可以看到使用+=操作符将一个字符串追加到了**s**上，并且使用了括号来控制表达式的赋值顺序，以使得**int**类型的变量在显示之前确实进行了求和操作。

请注意**main()**中的最后一个示例：有时会看到这种一个空的**String**后面跟随+和一个基本类型变量，以此作为不调用更加麻烦的显式方法（在本例中应该是**Integer.toString()**）而执行字符串转换的方式。

3.14 使用操作符时常犯的错误

使用操作符时一个常犯的错误就是，即使对表达式如何计算有点不确定，也不愿意使用括号。这个问题在Java中仍然存在。

在C和C++中，一个特别常见的错误如下：

```
while(x = y) {
  // ....
}
```

程序员很明显是想测试是否“相等”（==），而不是进行赋值操作。在C和C++中，如果y是一个非零值，那么这种赋值的结果肯定是true，而这样便会得到一个无穷循环。在Java中，这个表达式的结果并不是布尔值，而编译器期望的是一个布尔值。由于Java不会自动地将**int**数值转换成布尔值，所以在编译时会抛出一个编译时错误，从而阻止我们进一步去运行程序。所以这种错误在Java中永远不会出现（唯一不会得到编译时错误的情况是**x**和**y**都为布尔值。在这种情况下，**x = y**属于合法表达式。而在前面的例子中，则可能是一个错误）。

119

Java中有一个与C和C++中类似的问题，即使用按位“与”和按位“或”代替逻辑“与”和逻辑“或”。按位“与”和按位“或”使用单字符（**&**或**|**），而逻辑“与”和逻辑“或”使用双字符（**&&**或**||**）。就像“=”和“==”一样，键入一个字符当然要比键入两个简单。Java编译器可防止这个错误发生，因为它不允许我们随便把一种类型当作另一种类型来用。

3.15 类型转换操作符

类型转换（cast）的原意是“模型铸造”。在适当的时候，Java会将一种数据类型自动转换成另一种。例如，假设我们为某浮点变量赋以一个整数值，编译器会将**int**自动转换成**float**。类型转换运算允许我们显式地进行这种类型的转换，或者在不能自动进行转换的时候强制进行类型转换。

要想执行类型转换，需要将希望得到的数据类型置于圆括号内，放在要进行类型转换的值的左边，可以在下面的示例中看到它：

```
//: operators/Casting.java

public class Casting {
  public static void main(String[] args) {
    int i = 200;
```

```
    long lng = (long)i;
    lng = i; // "Widening," so cast not really required
    long lng2 = (long)200;
    lng2 = 200;
    // A "narrowing conversion":
    i = (int)lng2; // Cast required
  }
} ///:~
```

正如所看到的，既可对数值进行类型转换，亦可对变量进行类型转换。请注意，这里可能会引入“多余的”转型，例如，编译器在必要的时候会自动进行**int**值到**long**值的提升。但是你仍然可以做这样“多余的”事，以提醒自己需要留意，也可以使代码更清楚。在其他情况下，可能只有先进行类型转换，代码编译才能通过。

在C和C++中，类型转换有时会让人头痛。但是在Java中，类型转换则是一种比较安全的操作。然而，如果要执行一种名为*窄化转换*（narrowing conversion）的操作（也就是说，将能容纳更多信息的数据类型转换成无法容纳那么多信息的类型），就有可能面临信息丢失的危险。此时，编译器会强制我们进行类型转换，这实际上是说：“这可能是一件危险的事情，如果无论如何要这么做，必须显式地进行类型转换。”而对于*扩展转换*（widening conversion），则不必显式地进行类型转换，因为新类型肯定能容纳原来类型的信息，不会造成任何信息的丢失。 120

Java允许我们把任何基本数据类型转换成别的基本数据类型，但布尔型除外，后者根本不允许进行任何类型的转换处理。“类”数据类型不允许进行类型转换。为了将一种类转换成另一种，必须采用特殊的方法（本书后面会讲到，对象可以在其所属类型的类族之间可以进行类型转换；例如，“橡树”可转型为“树”；反之亦然。但不能把它转换成类族以外的类型，如“岩石”）。

3.15.1 截尾和舍入

在执行窄化转换时，必须注意截尾与舍入问题。例如，如果将一个浮点值转换为整型值，Java会如何处理呢？例如，将29.7转换为**int**，结果是30还是29？在下面的示例中可以找到答案：

```
//: operators/CastingNumbers.java
// What happens when you cast a float
// or double to an integral value?
import static net.mindview.util.Print.*;

public class CastingNumbers {
  public static void main(String[] args) {
    double above = 0.7, below = 0.4;
    float fabove = 0.7f, fbelow = 0.4f;
    print("(int)above: " + (int)above);
    print("(int)below: " + (int)below);
    print("(int)fabove: " + (int)fabove);
    print("(int)fbelow: " + (int)fbelow);
  }
} /* Output:
(int)above: 0
(int)below: 0
(int)fabove: 0
(int)fbelow: 0
*///:~
```

121

因此答案是在将**float**或**double**转型为整型值时，总是对该数字执行截尾。如果想要得到舍入的结果，就需要使用**java.lang.Math**中的**round()**方法：

```
//: operators/RoundingNumbers.java
// Rounding floats and doubles.
import static net.mindview.util.Print.*;

public class RoundingNumbers {
  public static void main(String[] args) {
```

```
    double above = 0.7, below = 0.4;
    float fabove = 0.7f, fbelow = 0.4f;
    print("Math.round(above): " + Math.round(above));
    print("Math.round(below): " + Math.round(below));
    print("Math.round(fabove): " + Math.round(fabove));
    print("Math.round(fbelow): " + Math.round(fbelow));
  }
} /* Output:
Math.round(above): 1
Math.round(below): 0
Math.round(fabove): 1
Math.round(fbelow): 0
*///:~
```

由于**round()**是**java.lang**的一部分，因此在使用它时不需要额外地导入。

3.15.2 提升

如果对基本数据类型执行算术运算或按位运算，大家会发现，只要类型比**int**小（即**char**、**byte**或者**short**），那么在运算之前，这些值会自动转换成int。这样一来，最终生成的结果就是**int**类型。如果想把结果赋值给较小的类型，就必须使用类型转换（既然把结果赋给了较小的类型，就可能出现信息丢失）。通常，表达式中出现的最大的数据类型决定了表达式最终结果的数据类型。如果将一个**float**值与一个**double**值相乘，结果就是**double**；如果将一个**int**和一个**long**值相加，则结果为**long**。

3.16 Java没有sizeof

在C和C++中，**sizeof()**操作符可以告诉你为数据项分配的字节数。在C和C++中，需要使用**sizeof()**的最大原因是为了“移植”。不同的数据类型在不同的机器上可能有不同的大小，所以在进行一些与存储空间有关的运算时，程序员必须获悉那些类型具体有多大。例如，一台计算机可用32位来保存整数，而另一台只用16位保存。显然，在第一台机器中，程序可保存更大的值。可以想像，移植是令C和C++程序员颇为头痛的一个问题。

122

Java不需要**sizeof()**操作符来满足这方面的需要，因为所有数据类型在所有机器中的大小都是相同的。我们不必考虑移植问题——它已经被设计在语言中了。

3.17 操作符小结

下面这个例子向大家展示了哪些基本数据类型能进行哪些特定的运算。基本上这是同一个不断重复的程序，只是每次使用了不同的基本数据类型。文件编译时不会报错，因为那些会导致编译失败的行已用//!注释掉了。

```
//: operators/AllOps.java
// Tests all the operators on all the primitive data types
// to show which ones are accepted by the Java compiler.

public class AllOps {
  // To accept the results of a boolean test:
  void f(boolean b) {}
  void boolTest(boolean x, boolean y) {
    // Arithmetic operators:
    //! x = x * y;
    //! x = x / y;
    //! x = x % y;
    //! x = x + y;
    //! x = x - y;
    //! x++;
    //! x--;
```

```
    //! x = +y;
    //! x = -y;
    // Relational and logical:
    //! f(x > y);
    //! f(x >= y);
    //! f(x < y);
    //! f(x <= y);
    f(x == y);
    f(x != y);
    f(!y);
    x = x && y;
    x = x || y;
    // Bitwise operators:
    //! x = ~y;
    x = x & y;
    x = x | y;
    x = x ^ y;
    //! x = x << 1;
    //! x = x >> 1;
    //! x = x >>> 1;
    // Compound assignment:
    //! x += y;
    //! x -= y;
    //! x *= y;
    //! x /= y;
    //! x %= y;
    //! x <<= 1;
    //! x >>= 1;
    //! x >>>= 1;
    x &= y;
    x ^= y;
    x |= y;
    // Casting:
    //! char c = (char)x;
    //! byte b = (byte)x;
    //! short s = (short)x;
    //! int i = (int)x;
    //! long l = (long)x;
    //! float f = (float)x;
    //! double d = (double)x;
  }
  void charTest(char x, char y) {
    // Arithmetic operators:
    x = (char)(x * y);
    x = (char)(x / y);
    x = (char)(x % y);
    x = (char)(x + y);
    x = (char)(x - y);
    x++;
    x--;
    x = (char)+y;
    x = (char)-y;
    // Relational and logical:
    f(x > y);
    f(x >= y);
    f(x < y);
    f(x <= y);
    f(x == y);
    f(x != y);
    //! f(!x);
    //! f(x && y);
    //! f(x || y);
    // Bitwise operators:
    x= (char)~y;
    x = (char)(x & y);
    x  = (char)(x | y);
    x = (char)(x ^ y);
```

123

124

```
    x = (char)(x << 1);
    x = (char)(x >> 1);
    x = (char)(x >>> 1);
    // Compound assignment:
    x += y;
    x -= y;
    x *= y;
    x /= y;
    x %= y;
    x <<= 1;
    x >>= 1;
    x >>>= 1;
    x &= y;
    x ^= y;
    x |= y;
    // Casting:
    //! boolean bl = (boolean)x;
    byte b = (byte)x;
    short s = (short)x;
    int i = (int)x;
    long l = (long)x;
    float f = (float)x;
    double d = (double)x;
  }
  void byteTest(byte x, byte y) {
    // Arithmetic operators:
    x = (byte)(x* y);
    x = (byte)(x / y);
    x = (byte)(x % y);
    x = (byte)(x + y);
    x = (byte)(x - y);
    x++;
    x--;
    x = (byte)+ y;
    x = (byte)- y;
    // Relational and logical:
    f(x > y);
    f(x >= y);
    f(x < y);
    f(x <= y);
    f(x == y);
    f(x != y);
    //! f(!x);
    //! f(x && y);
    //! f(x || y);
    // Bitwise operators:
    x = (byte)~y;
    x = (byte)(x & y);
    x = (byte)(x | y);
    x = (byte)(x ^ y);
    x = (byte)(x << 1);
    x = (byte)(x >> 1);
    x = (byte)(x >>> 1);
    // Compound assignment:
    x += y;
    x -= y;
    x *= y;
    x /= y;
    x %= y;
    x <<= 1;
    x >>= 1;
    x >>>= 1;
    x &= y;
    x ^= y;
    x |= y;
    // Casting:
    //! boolean bl = (boolean)x;
```

125

```
  char c = (char)x;
  short s = (short)x;
  int i = (int)x;
  long l = (long)x;
  float f = (float)x;
  double d = (double)x;
}
void shortTest(short x, short y) {
  // Arithmetic operators:
  x = (short)(x * y);
  x = (short)(x / y);
  x = (short)(x % y);
  x = (short)(x + y);
  x = (short)(x - y);
  x++;
  x--;
  x = (short)+y;
  x = (short)-y;
  // Relational and logical:
  f(x > y);
  f(x >= y);
  f(x < y);
  f(x <= y);
  f(x == y);
  f(x != y);
  //! f(!x);
  //! f(x && y);
  //! f(x || y);
  // Bitwise operators:
  x = (short)~y;
  x = (short)(x & y);
  x = (short)(x | y);
  x = (short)(x ^ y);
  x = (short)(x << 1);
  x = (short)(x >> 1);
  x = (short)(x >>> 1);
  // Compound assignment:
  x += y;
  x -= y;
  x *= y;
  x /= y;
  x %= y;
  x <<= 1;
  x >>= 1;
  x >>>= 1;
  x &= y;
  x ^= y;
  x |= y;
  // Casting:
  //! boolean bl = (boolean)x;
  char c = (char)x;
  byte b = (byte)x;
  int i = (int)x;
  long l = (long)x;
  float f = (float)x;
  double d = (double)x;
}
void intTest(int x, int y) {
  // Arithmetic operators:
  x = x * y;
  x = x / y;
  x = x % y;
  x = x + y;
  x = x - y;
  x++;
  x--;
  x = +y;
```

126

127

```
    x = -y;
    // Relational and logical:
    f(x > y);
    f(x >= y);
    f(x < y);
    f(x <= y);
    f(x == y);
    f(x != y);
    //! f(!x);
    //! f(x && y);
    //! f(x || y);
    // Bitwise operators:
    x = ~y;
    x = x & y;
    x = x | y;
    x = x ^ y;
    x = x << 1;
    x = x >> 1;
    x = x >>> 1;
    // Compound assignment:
    x += y;
    x -= y;
    x *= y;
    x /= y;
    x %= y;
    x <<= 1;
    x >>= 1;
    x >>>= 1;
    x &= y;
    x ^= y;
    x |= y;
    // Casting:
    //! boolean bl = (boolean)x;
    char c = (char)x;
    byte b = (byte)x;
    short s = (short)x;
    long l = (long)x;
    float f = (float)x;
    double d = (double)x;
  }
  void longTest(long x, long y) {
    // Arithmetic operators:
    x = x * y;
    x = x / y;
    x = x % y;
    x = x + y;
    x = x - y;
    x++;
    x--;
    x = +y;
    x = -y;
    // Relational and logical:
    f(x > y);
    f(x >= y);
    f(x < y);
    f(x <= y);
    f(x == y);
    f(x != y);
    //! f(!x);
    //! f(x && y);
    //! f(x || y);
    // Bitwise operators:
    x = ~y;
    x = x & y;
    x = x | y;
    x = x ^ y;
    x = x << 1;
```

128

```
    x = x >> 1;
    x = x >>> 1;
    // Compound assignment:
    x += y;
    x -= y;
    x *= y;
    x /= y;
    x %= y;
    x <<= 1;
    x >>= 1;
    x >>>= 1;
    x &= y;
    x ^= y;
    x |= y;
    // Casting:
    //! boolean bl = (boolean)x;
    char c = (char)x;
    byte b = (byte)x;
    short s = (short)x;
    int i = (int)x;
    float f = (float)x;
    double d = (double)x;
  }
  void floatTest(float x, float y) {
    // Arithmetic operators:
    x = x * y;
    x = x / y;
    x = x % y;
    x = x + y;
    x = x - y;
    x++;
    x--;
    x = +y;
    x = -y;
    // Relational and logical:
    f(x > y);
    f(x >= y);
    f(x < y);
    f(x <= y);
    f(x == y);
    f(x != y);
    //! f(!x);
    //! f(x && y);
    //! f(x || y);
    // Bitwise operators:
    //! x = ~y;
    //! x = x & y;
    //! x = x | y;
    //! x = x ^ y;
    //! x = x << 1;
    //! x = x >> 1;
    //! x = x >>> 1;
    // Compound assignment:
    x += y;
    x -= y;
    x *= y;
    x /= y;
    x %= y;
    //! x <<= 1;
    //! x >>= 1;
    //! x >>>= 1;
    //! x &= y;
    //! x ^= y;
    //! x |= y;
    // Casting:
    //! boolean bl = (boolean)x;
    char c = (char)x;
```

129

130

```
    byte b = (byte)x;
    short s = (short)x;
    int i = (int)x;
    long l = (long)x;
    double d = (double)x;
  }
  void doubleTest(double x, double y) {
    // Arithmetic operators:
    x = x * y;
    x = x / y;
    x = x % y;
    x = x + y;
    x = x - y;
    x++;
    x--;
    x = +y;
    x = -y;
    // Relational and logical:
    f(x > y);
    f(x >= y);
    f(x < y);
    f(x <= y);
    f(x == y);
    f(x != y);
    //! f(!x);
    //! f(x && y);
    //! f(x || y);
    // Bitwise operators:
    //! x = ~y;
    //! x = x & y;
    //! x = x | y;
    //! x = x ^ y;
    //! x = x << 1;
    //! x = x >> 1;
    //! x = x >>> 1;
    // Compound assignment:
    x += y;
    x -= y;
    x *= y;
    x /= y;
    x %= y;
    //! x <<= 1;
    //! x >>= 1;
    //! x >>>= 1;
    //! x &= y;
    //! x ^= y;
    //! x |= y;
    // Casting:
    //! boolean bl = (boolean)x;
    char c = (char)x;
    byte b = (byte)x;
    short s = (short)x;
    int i = (int)x;
    long l = (long)x;
    float f = (float)x;
  }
} ///:~
```

131

注意，能够对布尔型值进行的运算非常有限。我们只能赋予它**true**和**false**值，并测试它为真还是为假，而不能将布尔值相加，或对布尔值进行其他任何运算。

在**char**、**byte**和**short**中，我们可看到使用算术操作符中数据类型提升的效果。对这些类型的任何一个进行算术运算，都会获得一个**int**结果，必须将其显式地类型转换回原来的类型（窄化转换可能会造成信息的丢失），以将值赋给原本的类型。但对于**int**值，却不必进行类型转化，因为所有数据都已经属于**int**类型。但不要放松警惕，认为一切事情都是安全的，如果对两个足

132

够大的**int**值执行乘法运算，结果就会溢出。下面这个例子向大家展示了这一点：

```
//: operators/Overflow.java
// Surprise! Java lets you overflow.

public class Overflow {
  public static void main(String[] args) {
    int big = Integer.MAX_VALUE;
    System.out.println("big = " + big);
    int bigger = big * 4;
    System.out.println("bigger = " + bigger);
  }
} /* Output:
big = 2147483647
bigger = -4
*///:~
```

你不会从编译器那里收到出错或警告信息，运行时也不会出现异常。这说明Java虽然是好东西，但也没有那么好！

对于**char**、**byte**或者**short**，复合赋值并不需要类型转换。尽管它们执行类型提升，但也会获得与直接算术运算相同的结果。而在另一方面，省略类型转换可使代码更加简练。

可以看到，除**boolean**以外，任何一种基本类型都可通过类型转换变为其他基本类型。再一次提醒读者，当类型转换成一种较小的类型时，必须留意“窄化转换”的结果；否则会在类型转化过程中不知不觉地丢失了信息。

练习14：(3) 编写一个接收两个字符串参数的方法，用各种布尔值的比较关系来比较这两个字符串，然后把结果打印出来。做==和 != 比较的同时，用**equals()**作测试。在**main()**里面用几个不同的字符串对象调用这个方法。

3.18 总结

如果你拥有编程语言的经验，那么只要它的语法类似于C，你就会发现Java中的操作符与它们是多么地相似，以至于对你来说没有任何学习困难。如果你发现本章颇具挑战性，那么你应该先阅读从www.MindView.net上可获得的Thinking in C多媒体材料。

所选习题的答案都可以在名为The Thinking in Java Annotated Solution Guide的电子文档中找到，读者可以从www.MindView.net上购买此文档。 133~134